

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name: Srivatsan S

Reg No.: 24011101093

Experiment 1 Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link: <https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap

- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

7. Source Code

<https://github.com/sriwhatsnext/dllab/tree/main/Week-01-Perceptron>

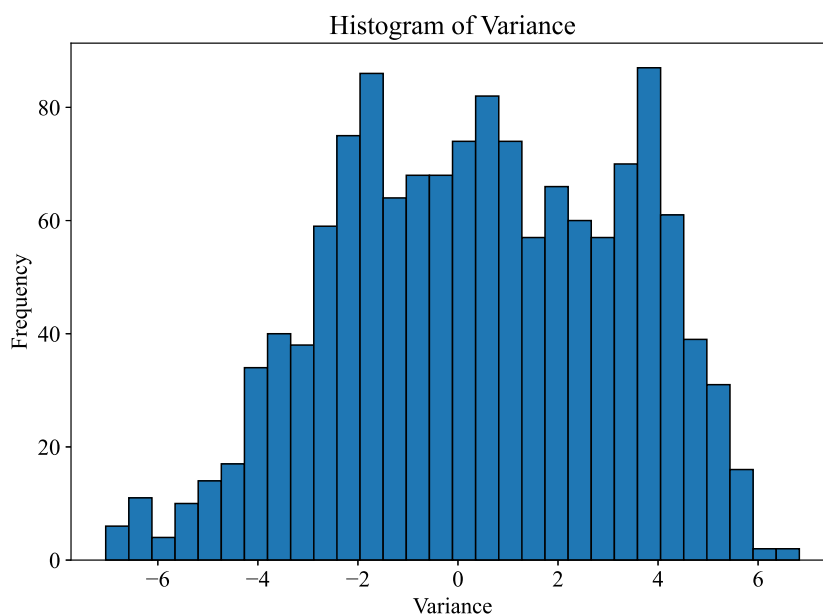
8. Experimental Results

The Single Layer Perceptron is implemented from scratch and trained on the Banknote Authentication dataset using the perceptron learning algorithm. The model was trained for 20 epochs with a learning rate of 0.1. During the training process, the number of misclassified samples went down with successive epochs, meaning that the perceptron was able to learn the underlying decision boundary and converge effectively.

The final trained model achieved an accuracy of 97.45 precision of 94.78 recall of 100.00 F1-score of 97.32 The confusion matrix showed 141 true negatives, 127 true positives, 7 false positives, and no false negatives, indicating that the model correctly classified almost all authentic and forged banknotes. The perceptron provides excellent performance for binary classification on the Banknote Authentication dataset, achieving high accuracy while correctly identifying all positive instances.

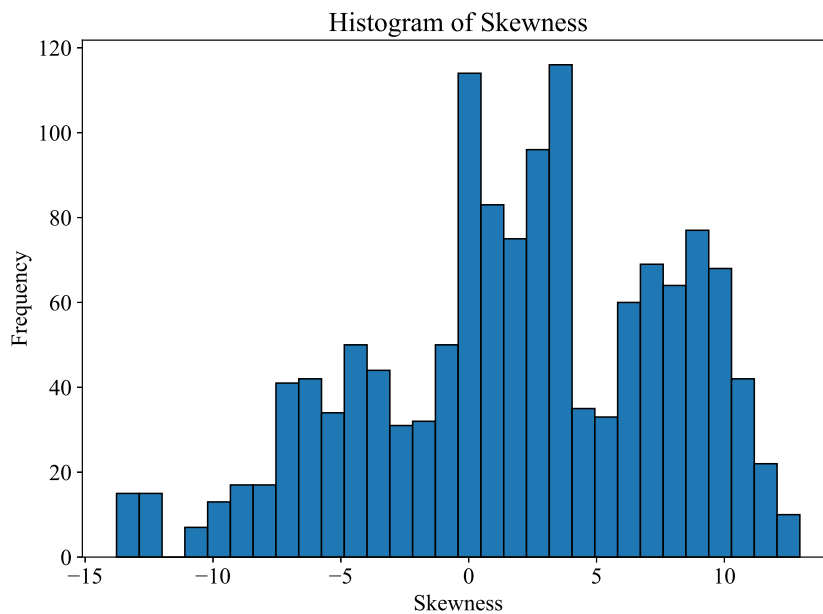
9. Generated Plots with Interpretation

9.1 Variance Histogram



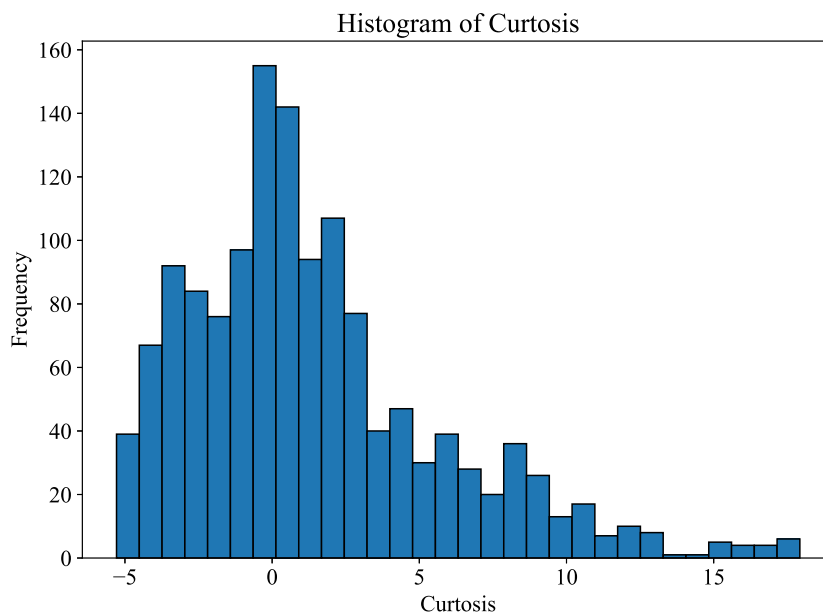
Interpretation: The distribution of the feature variance is almost normally spread with little skewed to the right.

9.2 Skewness Histogram



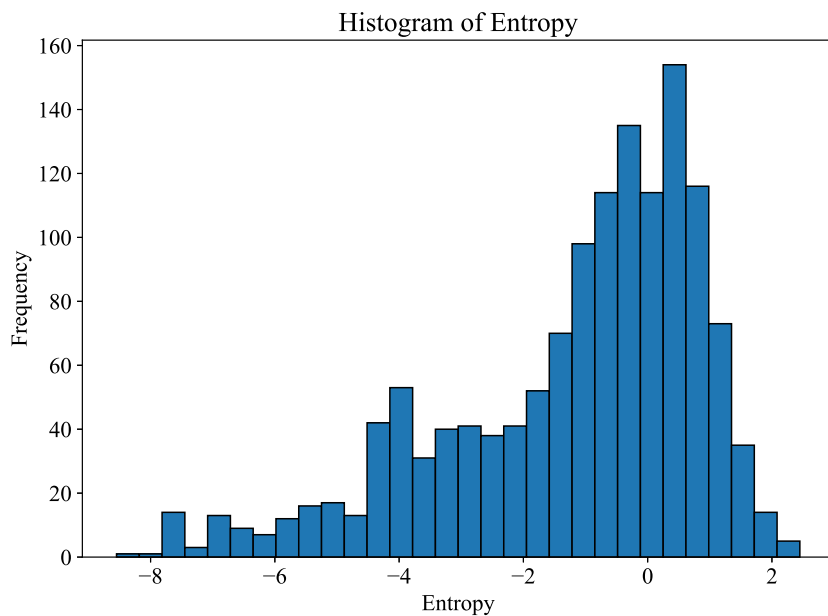
Interpretation: The distribution of the feature skewness is not uniform and has multiple peaks and also gaps with little skewed to the right side of 0 .

9.3 Kurtosis Histogram



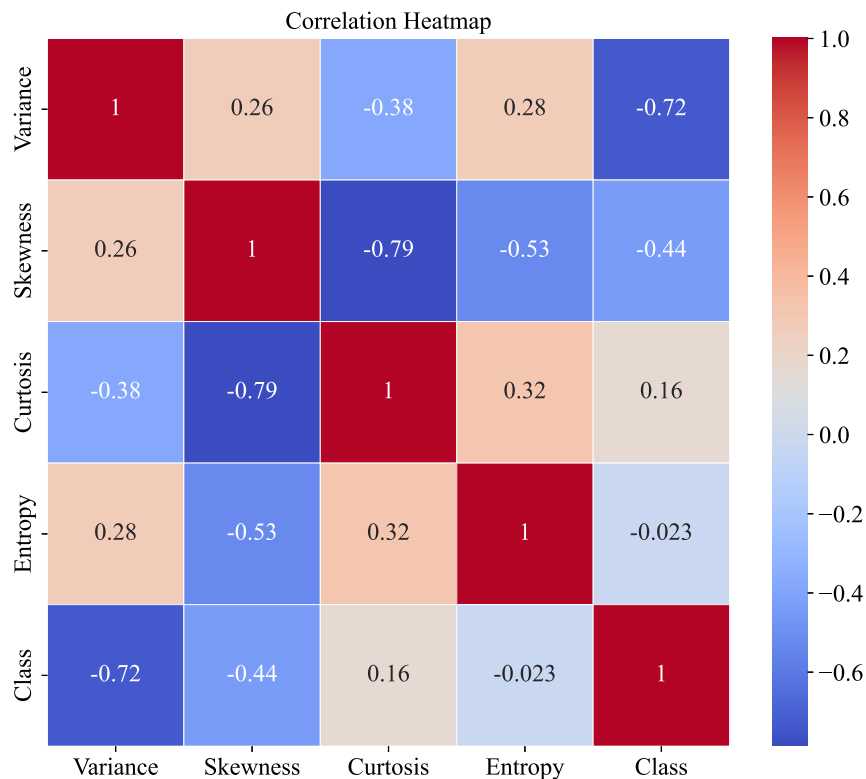
Interpretation: The graph is heavily skewed towards the left

9.4 Entropy Histogram



Interpretation: The graph is heavily skewed towards the right

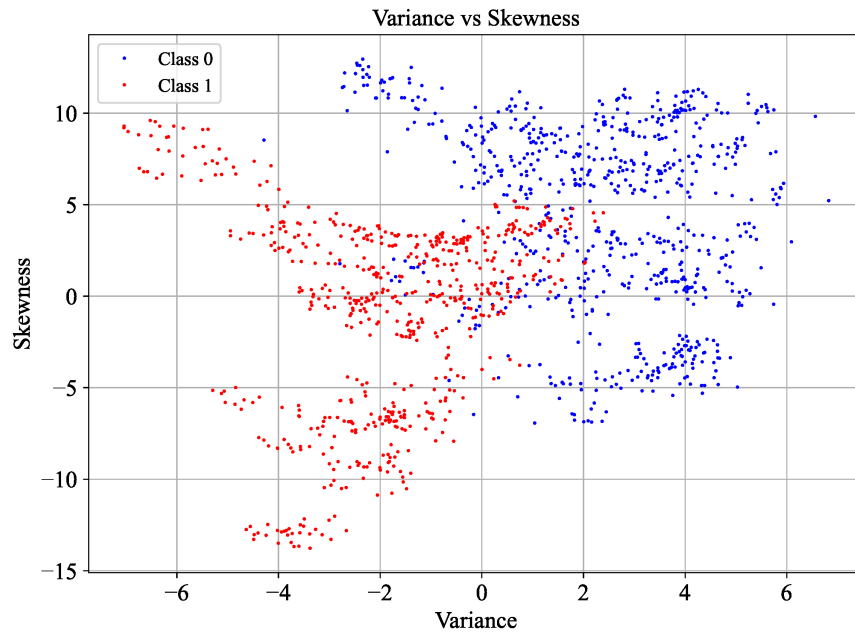
9.5 Correlation Heatmap



Interpretation: Each feature is completely correlated with itself (which is trivial) the more positively correlated features $\neq 0$ have direct proportionate correlation while more negative correlated

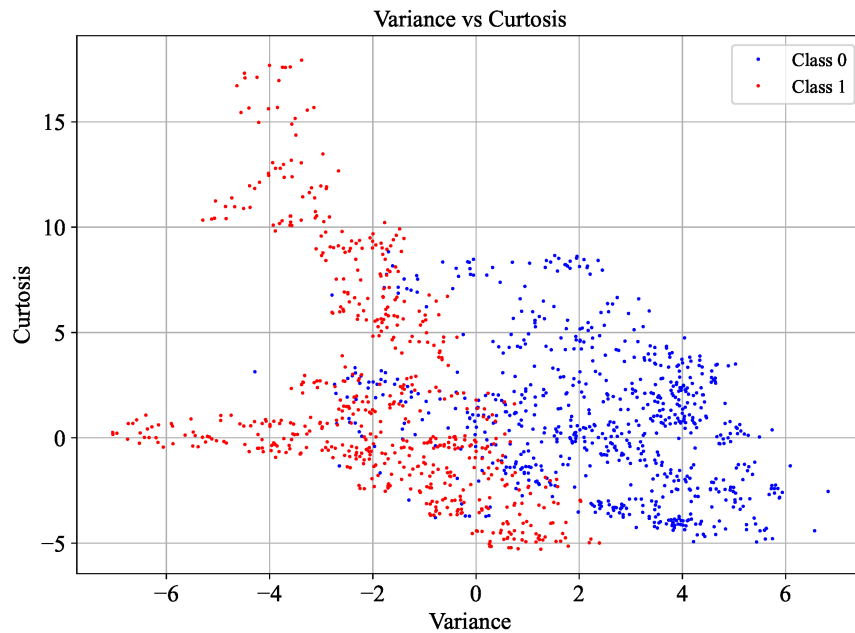
features are inversely proportionate . If two features have correlation as 0 they are completely uncorrelated .

9.6 Scatter Plot - Variance vs Skewness



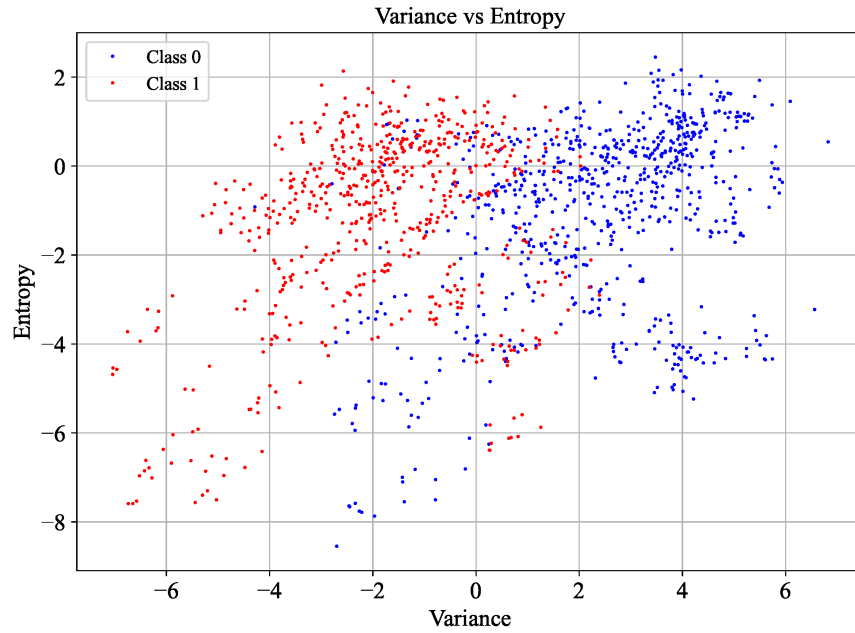
Interpretation: The data points have a small overlapping region and is fairly separated .

9.7 Scatter Plot - Variance vs Kurtosis



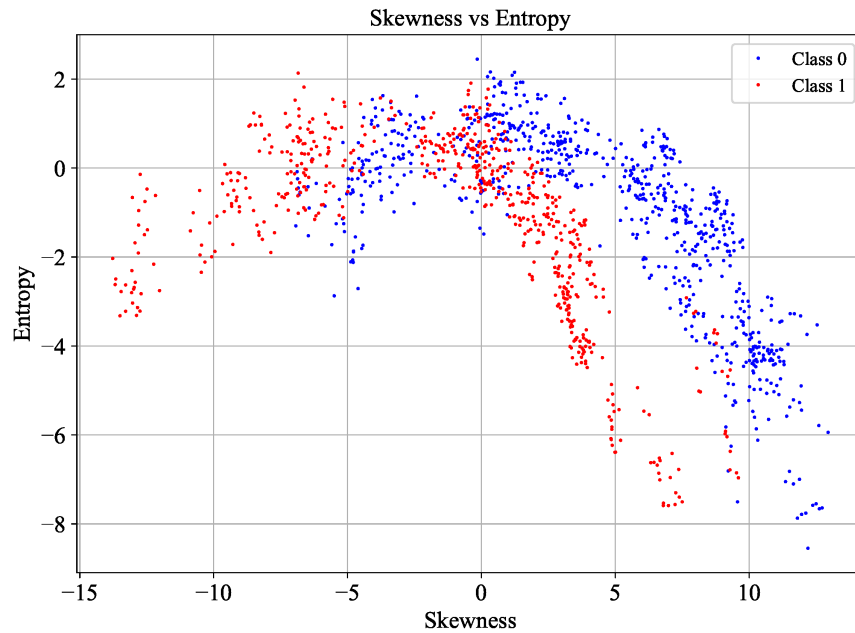
Interpretation: The data points are fairly separated with a small overlapping region and we can see it is not much correlated .

9.8 Scatter Plot - Variance vs Entropy



Interpretation: The data points are reasonably separated with moderate overlap and not much correlation.

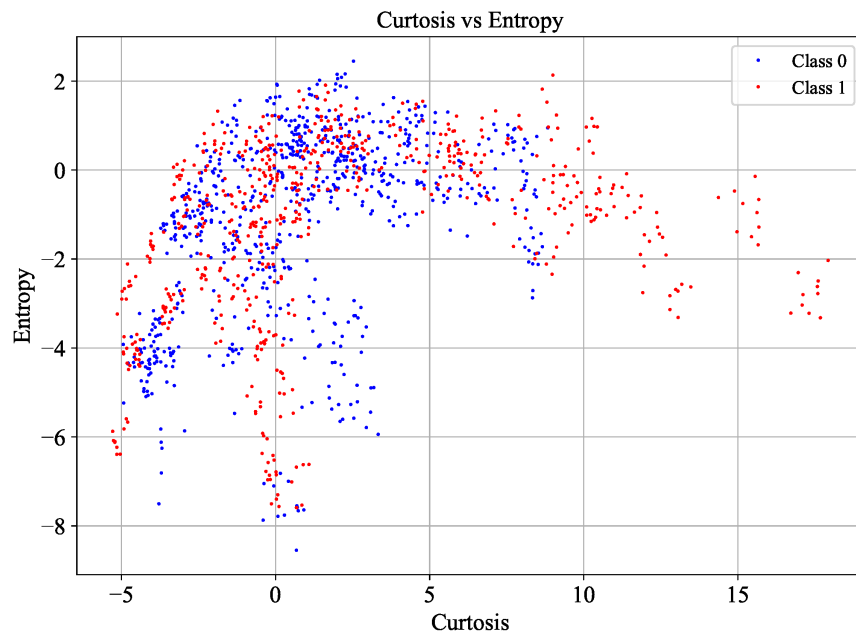
9.9 Scatter Plot - Skewness vs Entropy



Interpretation: The data points are reasonably separated with moderate overlap.

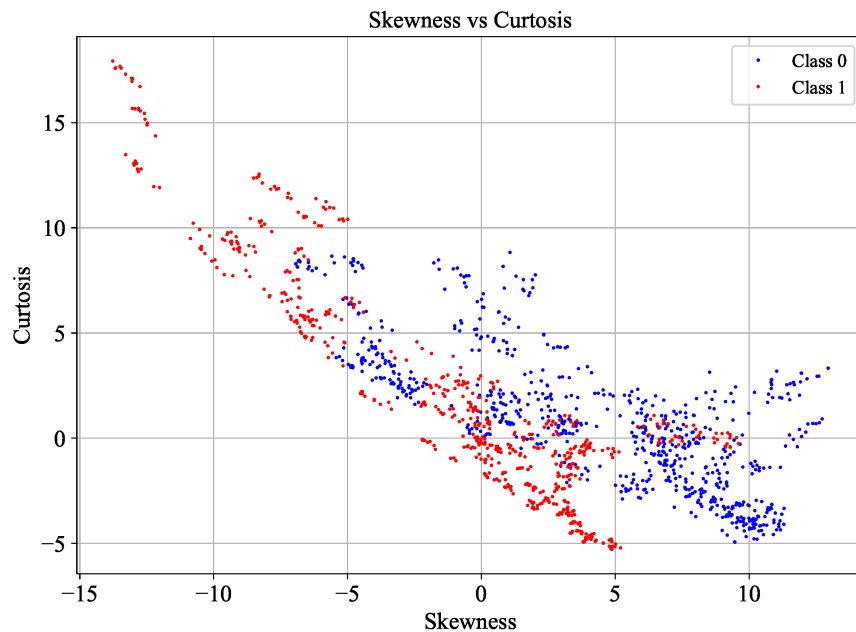
Interpretation: The data points are not that separated with a slightly higher overlapping region and we can see the negative correlation (negative slope).

9.10 Scatter Plot - Kurtosis vs Entropy



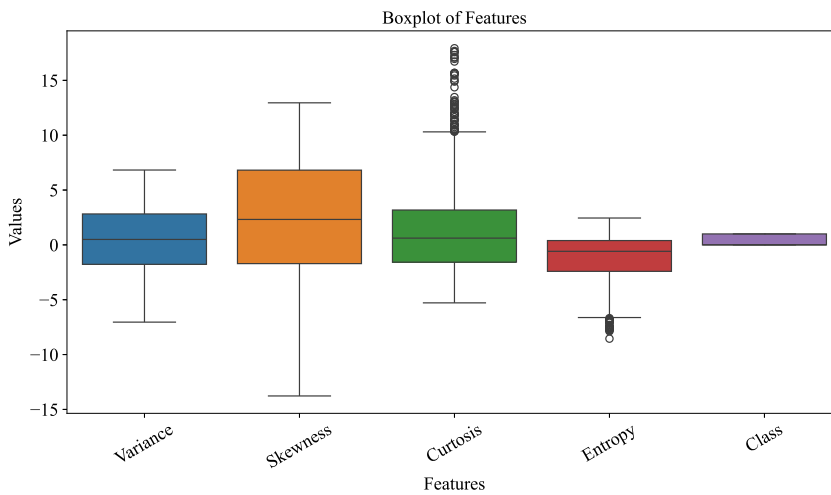
Interpretation: The data points are not fit for classification in this plane of space as it is completely overlapped .

9.11 Scatter Plot - Skewness vs Kurtosis



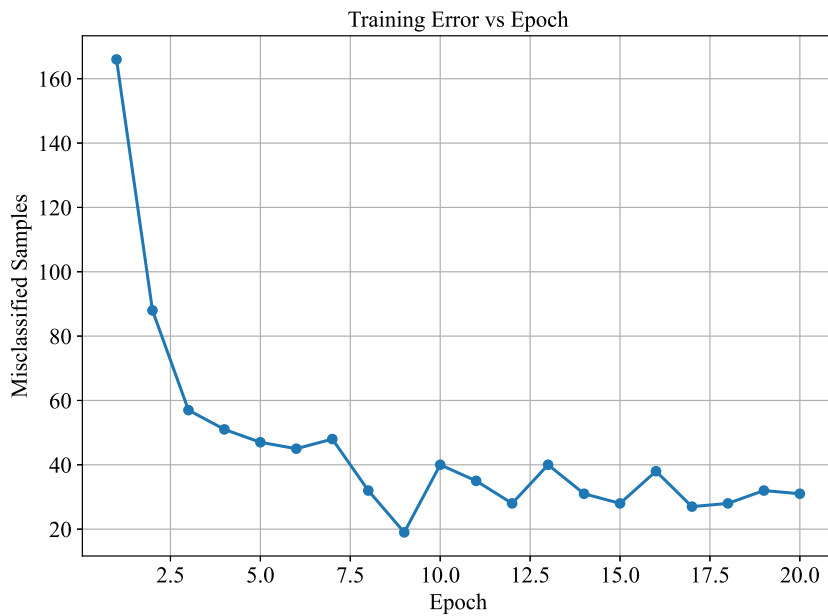
Interpretation: The data points still do not have a clean separation due to the overlapping

9.12 BOXPLOTS



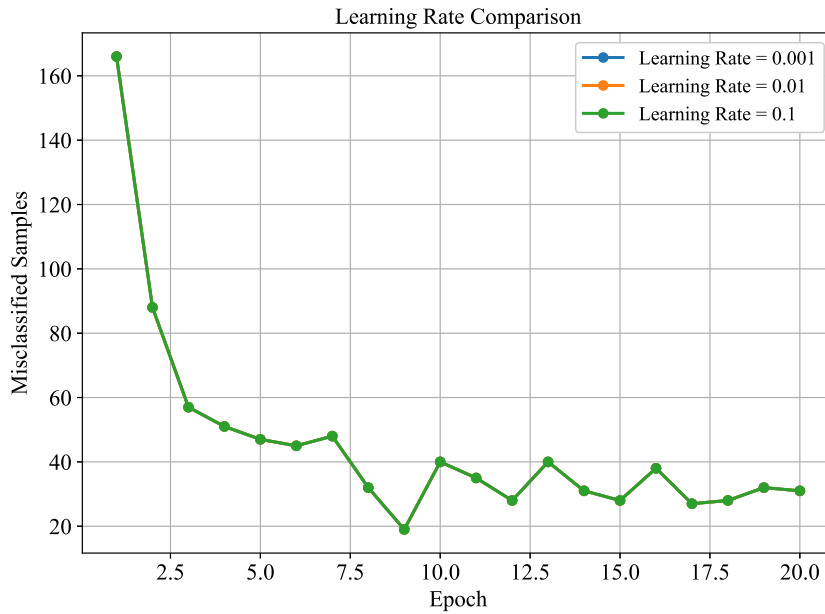
Interpretation: The features variance, skewness, Kurtosis have a fair separation but entropy is slightly skewed to the right. We can see outliers present in kurtosis and entropy features.

9.13 Training Error vs Epoch (Learning Rate = 0.1)



Interpretation: Training error decreases exponentially with the number of epochs and then fluctuates until it stabilizes.

9.14 Learning Rate Comparison



Interpretation: All the 3 learning rates converge at the same trend with the number of epochs because the dataset's points are linearly separable. With each misclassified point only the magnitude of the weight and bias updates change but the number of errors remain same. Hence the curve is the same for all learning rates

10. Performance Tables

Training Summary

Dataset Size	1372
Train/Test Split	80 / 20
Learning Rate	0.1
Epochs	20
Final Weights	[-1.46411,-1.32209,-1.52145,0.16122]
Final Bias	1.9000
Accuracy	0.9754
Precision	0.9477
Recall	1.0
F1-score	0.9731

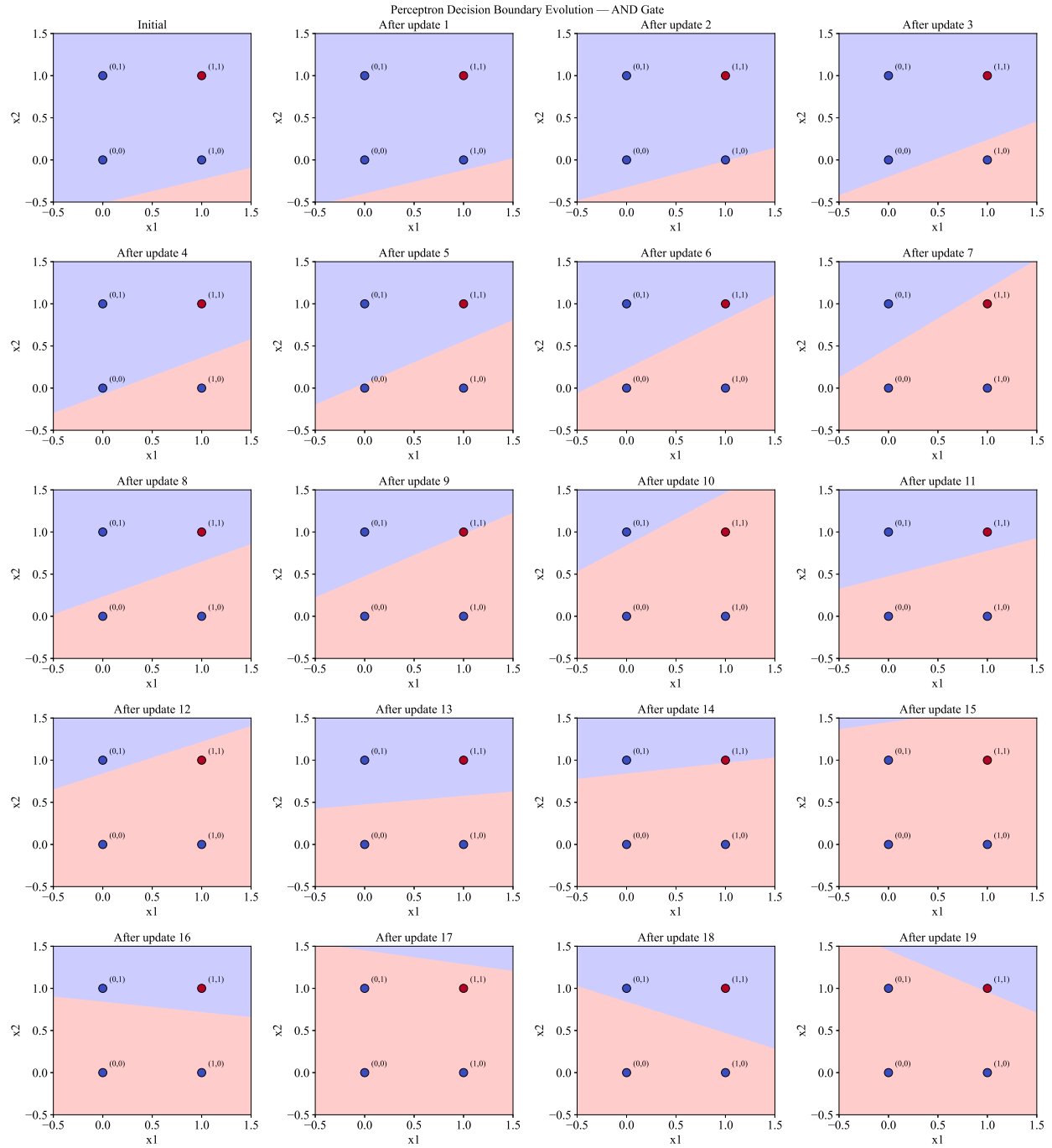
Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Weight 3	Weight 4	Bias
1	166	-0.682741	-0.386685	-0.531910	0.152935	0.8000
2	88	-0.831161	-0.614632	-0.768784	0.116242	1.0000
3	57	-0.873157	-0.656554	-0.959305	0.150600	1.1000
4	51	-0.953857	-0.797431	-0.928540	0.101702	1.2000
5	47	-0.977834	-0.856403	-1.002551	0.057811	1.3000
6	45	-1.058116	-0.886184	-1.014938	0.043303	1.4000
7	48	-1.116004	-0.991264	-1.083002	0.090912	1.4000
8	32	-1.179650	-1.012441	-1.161600	0.099908	1.4000
9	19	-1.153475	-1.008245	-1.156251	0.133772	1.5000
10	40	-1.228593	-1.050546	-1.208598	0.102051	1.5000
11	35	-1.222776	-1.108719	-1.209587	0.144226	1.6000
12	28	-1.274819	-1.115410	-1.263071	0.088048	1.6000
13	40	-1.338981	-1.164617	-1.328936	0.168120	1.6000
14	31	-1.363689	-1.132260	-1.345786	0.138184	1.7000
15	28	-1.399098	-1.170674	-1.372022	0.125264	1.7000
16	38	-1.421801	-1.236861	-1.449340	0.165241	1.7000
17	27	-1.401861	-1.205542	-1.475219	0.135846	1.8000
18	28	-1.451341	-1.264691	-1.462473	0.125418	1.8000
19	32	-1.490030	-1.301896	-1.519442	0.086041	1.8000
20	31	-1.464116	-1.322097	-1.521451	0.161220	1.9000

11. Additional Tasks

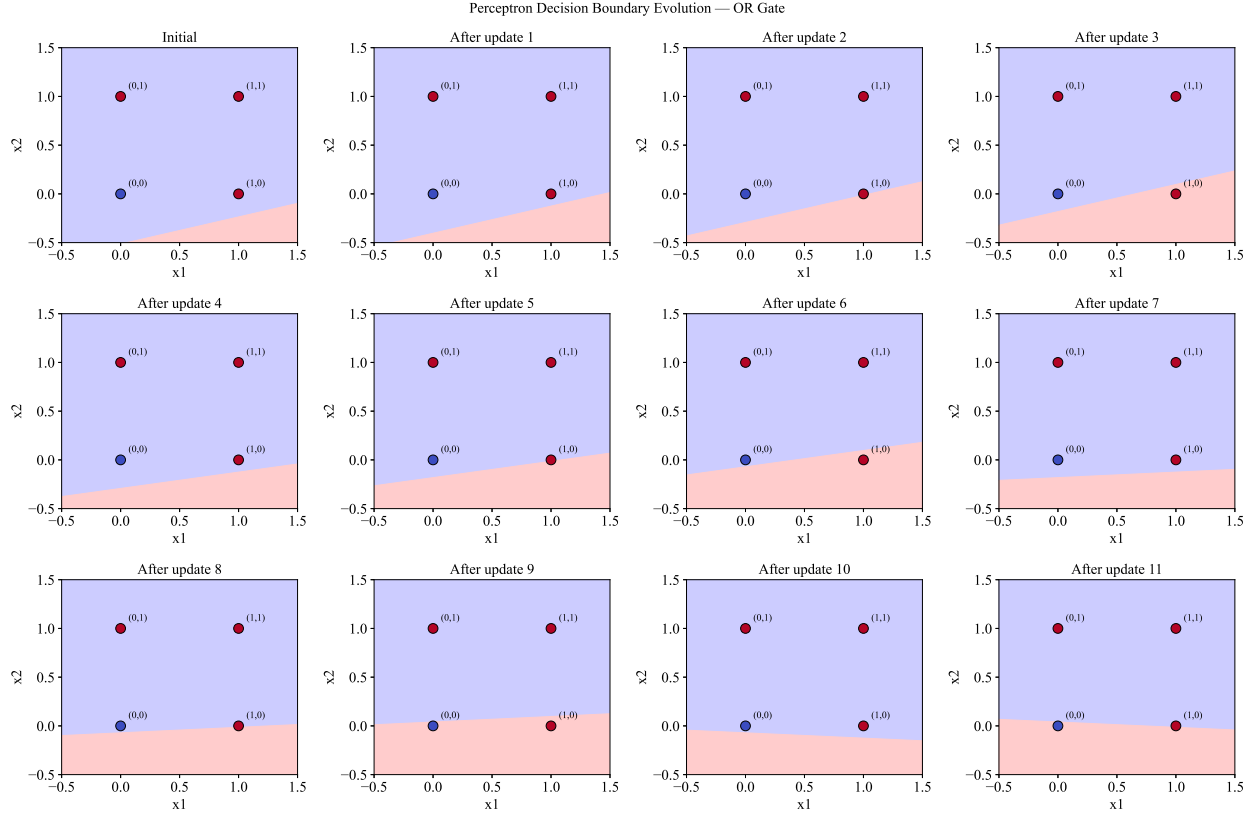
Code the perceptron learning algorithm for OR, NOT, and AND gates using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python built-in packages.

11.1 Perceptron And problem



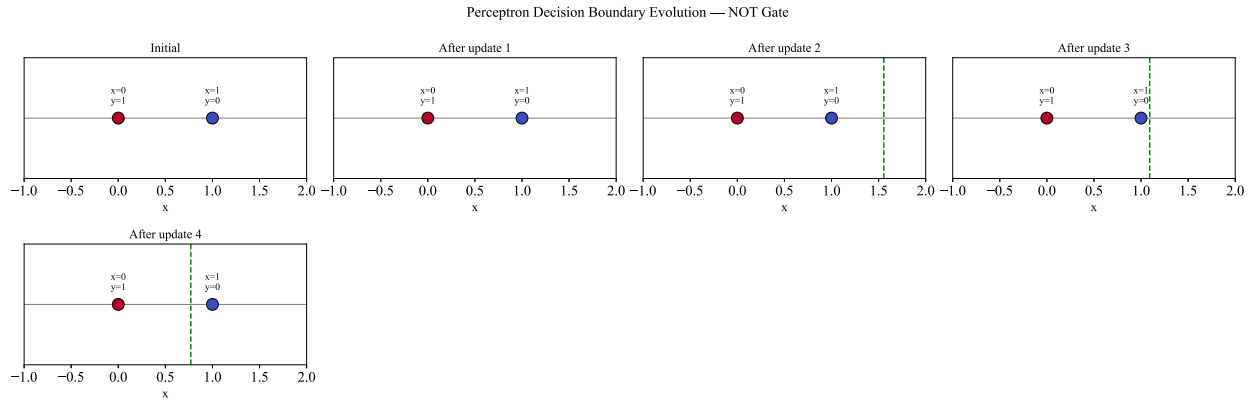
Interpretation: The decision boundary for the and gate has converged at the end of 19 epochs of a single layer perceptron

11.2 Perceptron OR problem



Interpretation: The decision boundary for the or gate has converged at the end of 11 epochs of a single layer perceptron

11.3 Perceptron NOT problem



Interpretation: The decision boundary for the not gate has converged at the end of 4 epochs of a single layer perceptron

12. Discussion

The perceptron algorithm was implemented and evaluated on the Banknote Authentication dataset . BY using learning rates of 0.001, 0.01, and 0.1. It was observed that the training error decreased in a similar manner across all learning rates, but the magnitude of weight and bias updates differed . By scaling all numerical features to a common range , no individual features dominated the update process . Consequently the perceptron achieved performance accuracy of 97.45, precision of 94.78, recall of 100, and an F1-score of 97.32

13. Conclusion

The experiments demonstrate that the perceptron algorithm is highly effective for binary classification with linearly separable data such as the banknote authentication dataset and the And Or Not Problems.

14. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.